

A Multi-Agent RAG Architecture with DPO Fine-Tuning for Automated Bug Report Generation

ARTICLE INFO

Keywords:

Bug Report Generation
Large Language Models
Multi-Agent System
Direct Preference Optimization (DPO)
Retrieval-Augmented Generation (RAG)
Software Maintenance
Reproducibility

ABSTRACT

CONTEXT

Bug reports are essential for software maintenance, yet many reports remain incomplete, ambiguous, or inconsistently structured. Existing automated approaches based on Large Language Models (LLMs) often rely on single-pipeline supervised fine-tuning (SFT), which struggles to ensure structural completeness, contextual fidelity, and reproducibility. These limitations hinder both defect diagnosis and large-scale maintenance workflows.

OBJECTIVES

This study aims to develop a reproducible and maintainable bug report generation framework that overcomes the methodological limitations of SFT and the architectural constraints of monolithic pipelines. The goal is to improve structural consistency, lexical coverage, and semantic coherence while providing a modular and experimentally reproducible system.

METHODS

We propose a Multi-Agent Retrieval-Augmented Generation (RAG) architecture combined with Direct Preference Optimization (DPO). The pipeline is decomposed into independent agents for data processing, prompt construction, training, generation, evaluation, and reporting, all orchestrated by a deterministic controller. The Qwen2.5-7B model is first trained using SFT and then optimized with DPO using automatically constructed preference pairs. RAG-based few-shot prompting retrieves semantically similar examples to enhance contextual alignment during generation. Experiments are conducted on 3,966 summary-ground truth pairs from Bugzilla.

RESULTS

The proposed DPO model achieves 81.1% CTQRS, 73.2% ROUGE-1 Recall, and 85.0% SBERT, outperforming the SFT baseline (77.0%, 61.0%, 85.0%) and GPT-4o. Adding RAG-based 1-shot prompting further improves performance to 83.2% CTQRS and 82.2% ROUGE-1 Recall. Results show consistent improvements across structural, lexical, and semantic dimensions and show stable behavior under bootstrap-based robustness analysis.

CONCLUSION

Integrating DPO with a modular multi-agent architecture significantly enhances the quality and reproducibility of automated bug report generation. The framework reduces structural omissions, strengthens contextual fidelity, and maintains semantic coherence, offering a practical and extensible solution for large-scale software maintenance workflows.

1. Introduction

Bug reports are essential artifacts within the software development lifecycle, as they document the root causes of defects and facilitate the tracking of corrective actions. However, in open-source projects, contributors with varying levels of expertise often compose reports in free-text formats, which frequently leads to missing information or vague descriptions. Such unstructured reports make defect reproduction and root cause analysis more difficult, ultimately reducing both maintenance efficiency and software quality.

Prior studies [1]–[3] have similarly reported that many bug reports contain incomplete or ambiguous information, which delays issue resolution. This problem is not limited to open-source environments; commercial issue-tracking systems such as Jira [4] also exhibit comparable patterns, and automated report structuring has become an important challenge across the broader field of software maintenance.

Recent advances in large language models (LLMs) [5], [6] have introduced new opportunities to address these challenges. LLMs possess strong contextual reasoning and natural language generation capabilities due to large-scale training, and techniques such as prompt engineering and fine-tuning allow the models to generate documents aligned with domain-specific constraints. However, prior approaches that rely on template-based generation or straightforward supervised fine-tuning (SFT) often struggle to achieve both structural completeness and contextual consistency [7]. In addition, few studies have quantitatively examined the impact of few-shot prompts [5], and the effect of preference alignment

methods such as Direct Preference Optimization (DPO) [8] on bug report quality has not been sufficiently explored.

To address these limitations, this study introduces a DPO-based multi-agent Retrieval-Augmented Generation (RAG) framework [9]–[13]. The proposed framework decomposes data preprocessing, prompt construction, training, report generation, evaluation, and result summarization into independent agents, enabling each stage to operate autonomously while maintaining cooperative interactions. Unlike single-pipeline designs, this modular structure simplifies component replacement and enhancement, providing both maintainability and reproducibility from a software engineering perspective. While existing LLM-based systems typically center on a single generator, the framework in this study adopts a collaborative agent architecture where the training, generation, and evaluation stages cross-reference one another, establishing a distinct architectural contribution.

The RAG-based approach [13] retrieves examples that are semantically similar to the input and incorporates them into the few-shot prompt, improving contextual consistency and information completeness. DPO is then applied to align the model outputs with preferred structural and narrative characteristics. This combined DPO–RAG design enhances contextual reasoning and logical coherence beyond what can be achieved through SFT alone, yielding improvements in structural, lexical, and semantic dimensions of automated report generation.

Experiments were conducted using 3,966 summary–ground truth pairs derived from Bugzilla datasets [14], [15]. The preference pairs required for DPO training were constructed without external data by comparing outputs from a baseline SFT model with the corresponding ground truth reports. This ensures a fair comparison between the proposed model and the baseline, as the only difference lies in the learning methodology.

The experimental results show that the proposed framework consistently achieves superior performance across structural, lexical, and semantic metrics compared to both SFT-based models and GPT-4o [6]. These improvements were repeatedly reproduced across multiple trials, indicating that the combination of DPO training, RAG-based prompting, and multi-agent architecture provides comprehensive enhancements in the reliability and quality of bug report generation.

The contributions of this study are summarized as follows:

- Proposal of a reproducible multi-agent architecture: By modularizing every stage from data processing to evaluation into independent agents, the framework strengthens both maintainability and reproducibility of the bug report generation pipeline.
- Quality improvements through DPO–RAG integration and a fair evaluation methodology: By combining preference alignment (DPO) with retrieval-based prompting (RAG) and constructing preference pairs from baseline model outputs and ground truth reports, the framework achieves higher structural completeness and contextual consistency than conventional SFT.

This paper is organized as follows. Section II reviews background knowledge and prior work on automated bug reporting. Section III describes the overall architecture of the proposed multi-agent framework and details the roles of each agent. Section IV presents the experimental setup, datasets, and evaluation metrics, followed by comparative analysis between the proposed model and the baseline. Section V discusses the implications and limitations of the results, and Section VI addresses the conclusion and potential directions for future research.

2. Background

2.1. Variability in Bug Report Quality and the Importance of Structured Reporting

Bug reports are essential artifacts in software maintenance because they support understanding and resolving defects. In practice, however, the quality of these reports varies widely depending on the reporter’s experience, writing habits, and familiarity with reporting tools [1]. This variability is especially prominent in open-source environments where contributors come from diverse backgrounds. As a result, critical elements such as steps to reproduce, execution environment, expected behavior, and actual behavior are often missing or vaguely described. Such incomplete reports hinder defect reproduction, require additional validation during analysis, and ultimately lead to delays in resolution and increased maintenance costs. Figure 1 compares two bug reports of differing quality and illustrates how they influence maintenance efficiency. The incomplete report in (a) lacks environment details and reproduction steps, and its expected behavior is unclear, creating obstacles for developers from the earliest stage of defect analysis. In contrast, the structured report in (b) provides all necessary information, including reproduction steps, environment settings, and expected behavior, which allows developers to reproduce the issue quickly and perform maintenance tasks more efficiently.

Incomplete Report (Low-Quality Example)	Structured Report (High-Quality Example)
Title: Shopping cart isn't working Body: I clicked the 'Quick Add' button to buy an item, but the cart is broken. The quantity is showing up weird. Please fix this ASAP.	Title: [UI] Cart quantity not updating on consecutive 'Quick Add' clicks Environment: - OS: Windows 11 - Browser: Chrome (v120.0.1) - App Version: v2.5.1 Steps to Reproduce: 1. Navigate to the 'Best Sellers' section on the main page (/home). 2. Click the 'Quick Add' button for the 'Blue Tumbler' item once. 3. (Result) The mini-cart icon in the top-right corner displays a quantity of '1'. 4. Without leaving the page, immediately click the 'Quick Add' button for the same 'Blue Tumbler' item a second time. Actual Result: - The mini-cart icon in the top-right corner still displays a quantity of '1'. Expected Result: - After the second click, the mini-cart icon quantity should update to '2'.
(a)	(b)

Figure 1. Contrast in bug report structure and completeness: (a) An unstructured, low-quality report lacking the necessary environment details, reproduction steps, and expected results. (b) A well-structured, high-quality report that clearly specifies the environment, steps to reproduce, and the expected outcome, enabling efficient debugging.

These differences clearly highlight the contrast between well-structured reports, which guide developers through precise procedures, environment configurations, and expected outcomes, and poorly written reports, which delay analysis by omitting essential information and introducing ambiguity. This contrast shows that the structural completeness of a report is closely tied to maintenance efficiency.

Several metrics have been used to quantitatively evaluate bug report quality. CTQRS measures structural completeness by assessing how thoroughly a report includes the core information required for defect resolution [16]. ROUGE evaluates lexical overlap with a ground-truth report to determine whether key terms are appropriately included [17]. SBERT-based metrics use sentence embeddings to assess semantic alignment with the ground-truth report [18]. Together, these metrics enable multidimensional evaluation of structural fidelity, lexical coverage, and semantic consistency.

However, quality evaluation alone cannot fully address the challenges encountered in real projects. Even when low-quality reports are identified, developers must still manually revise or restructure them [19]–[21]. This challenge becomes particularly significant in large-scale projects with a continuous influx of reports, where diagnostic approaches alone are insufficient to manage quality variability. To overcome this limitation, recent studies have expanded beyond evaluation and explored natural language processing (NLP) and large language models (LLMs) to automatically refine and structure incomplete bug reports [7], [22], [23]. This direction offers the potential to reduce quality disparities, accelerate defect resolution, and improve overall maintenance productivity.

2.2. Approaches and Limitations in Automated Bug Report Research

A variety of automation techniques have been proposed to address persistent quality issues in bug reports. Early approaches frequently relied on template-driven methods. Well-known issue tracking systems such as Bugzilla [15] required reporters to complete predefined forms, which partially mitigated the omission of essential fields such as reproduction steps or expected results. Although this strategy helped reduce variation in report quality, it often lacked flexibility for project-specific requirements and constrained the reporter’s autonomy. As a result, reports commonly achieved formal consistency but did not provide the level of contextual richness needed for effective maintenance activities.

Subsequent studies explored automation using machine learning (ML) techniques. For instance, text classification models were employed to detect duplicate bug reports or infer overall quality levels [19]–[21]. These methods offered the advantage of learning structural patterns in a data-driven manner and identifying report quality automatically. However, their performance degraded significantly when sufficient training data was unavailable. The lack of domain-specific datasets also limited generalizability, which meant that models often performed well on one project but could not be easily transferred to others. Furthermore, because these approaches primarily focused on classification or prediction, they did not provide mechanisms to directly structure a report or supply missing information.

The emergence of large language models (LLMs) has introduced a major shift in automated bug report research. Advanced LLMs, such as the GPT family, have achieved strong improvements in natural language understanding and generation by training on large-scale text corpora [5], [6]. Through prompt engineering and fine-tuning, LLMs can now be adapted to specialized tasks such as software report generation [24]. Several studies have experimented with using LLMs to summarize incomplete reports or fill in missing fields, and these attempts have showed

substantially greater potential than template- or ML-based approaches [7], [22], [23].

Despite these advances, most LLM-based studies still rely on a single model and a single processing pipeline [7]. This dependence makes it difficult to ensure consistent outputs for identical inputs and frequently requires complete pipeline redesigns when new evaluation metrics or strategies must be incorporated. Architectural constraints remain with respect to reproducibility, modularity, and scalability. In addition, because many of these systems rely solely on supervised fine-tuning (SFT), they experience methodological limitations in achieving reliable structural completeness and contextual coherence, as noted in the introduction.

Research on automated bug reporting has therefore progressed from template-based methods to ML-based classification and prediction, and more recently to LLM-based generation and completion. Although these developments represent meaningful progress, generating high-quality reports that are consistent, scalable, and dependable across diverse environments remains an unresolved challenge. These architectural and methodological gaps identified in the literature highlight the need for modular agent-based pipelines [9]–[12] and advanced training strategies such as DPO [8], which serve as the primary motivation for the framework proposed in this study.

2.3. Analysis of Existing Baselines and Identified Gaps

Recent studies on automating bug reports with large language models (LLMs) have made notable progress by moving beyond traditional templates or machine-learning-based approaches. Acharya and Ginde [7], for example, applied supervised fine-tuning (SFT) to open-source LLMs such as Qwen 2.5 [25] and Mistral [26], and both qualitative and quantitative analyses showed that these models can reach or exceed the level of structural completeness achieved by commercial systems like ChatGPT. These findings indicate that LLMs trained on sufficiently rich datasets can effectively adhere to the structured format of bug reports while inferring missing information.

Despite these advances, SFT-based baseline approaches still face fundamental challenges in terms of training methodology and architectural design. First, from a methodological perspective, plain SFT has limitations when learning what constitutes a higher-quality report. Because SFT focuses on modeling the text distribution of ground truth data, the model learns the shape of the correct answer but does not explicitly internalize which responses are more logical or more useful. Bug reports require more than a list of facts; they demand clarity in the causal relationship between reproduction steps and outcomes, and they must remain concise. With SFT alone, the model may produce grammatically correct outputs that are unnecessarily verbose or contain reasoning gaps, and these issues cannot be fully controlled. Techniques that capture relative quality differences, such as Direct Preference Optimization (DPO), can guide the model to recognize and reflect content-level superiority in addition to structural accuracy [8]. Second, from an architectural standpoint, the single-pipeline structure used in prior studies limits scalability and reproducibility. Baseline models often rely on a monolithic process that couples data processing and generation, which makes it difficult to modify or replace only certain components. General-purpose multi-agent frameworks like ReAct [10], AutoGen [11], and LangChain [12] may offer potential alternatives, but they do not fully align with the goals of this research. These frameworks emphasize autonomous communication and dynamic path exploration among agents, which can introduce non-deterministic behavior where results differ across runs. This variability becomes a risk factor in environments that require strict reproducibility or consistent quality control.

To address these limitations, this work proposes an integrated framework that combines preference-based training (DPO), which extends beyond imitation learning in SFT, with a deterministic multi-agent architecture that avoids the uncertainty found in general-purpose agent systems. This design is expected to enhance the logical quality of generated reports while ensuring modularity and reproducibility within the system.

3. Methodology

This section describes the architecture and core operational principles of the proposed multi-agent framework based on Direct Preference Optimization (DPO) [8] and Retrieval-Augmented Generation (RAG) [13].

The framework is designed to address the two limitations identified in Section 2: architectural limitations (lack of modularity and reproducibility) and methodological limitations (quality issues associated with SFT-based approaches). To overcome these constraints, the entire process is modularized into multiple agents, each responsible for a dedicated function. Unlike a single-pipeline approach, this design allows individual components to be replaced or improved with ease, which strengthens maintainability and scalability from a software engineering perspective.

Figure 2 illustrates the high-level architecture of the proposed framework. The overall workflow is orchestrated by the Controller Agent, which coordinates every stage of the process including data preparation, model training, report generation, and final evaluation. This centralized control ensures a consistent execution flow.

The main mechanisms for improving output quality are implemented by the Prompt Agent and the Training Agent. The Prompt Agent uses a RAG-based method [13] to retrieve cases similar to the given summary and construct a dynamic few-shot prompt that enhances contextual consistency. The Training Agent fine-tunes a pretrained language model (Qwen2.5–7B Instruct) [25] with LoRA (16-bit) [24], then performs additional training with DPO [8] so that the model aligns with human-preferred structural and narrative characteristics.

In addition to these core agents, the Data Agent, Generation Agent, Evaluation Agent, and Reporting Agent serve as supporting modules responsible for data preprocessing, report generation, automated quality assessment, and result aggregation. This multi-stage agent design offers a high degree of flexibility and extensibility, enabling each module to be independently enhanced.

The following sections describe the specific roles, internal mechanisms, and interactions of each agent that constitutes the architecture.

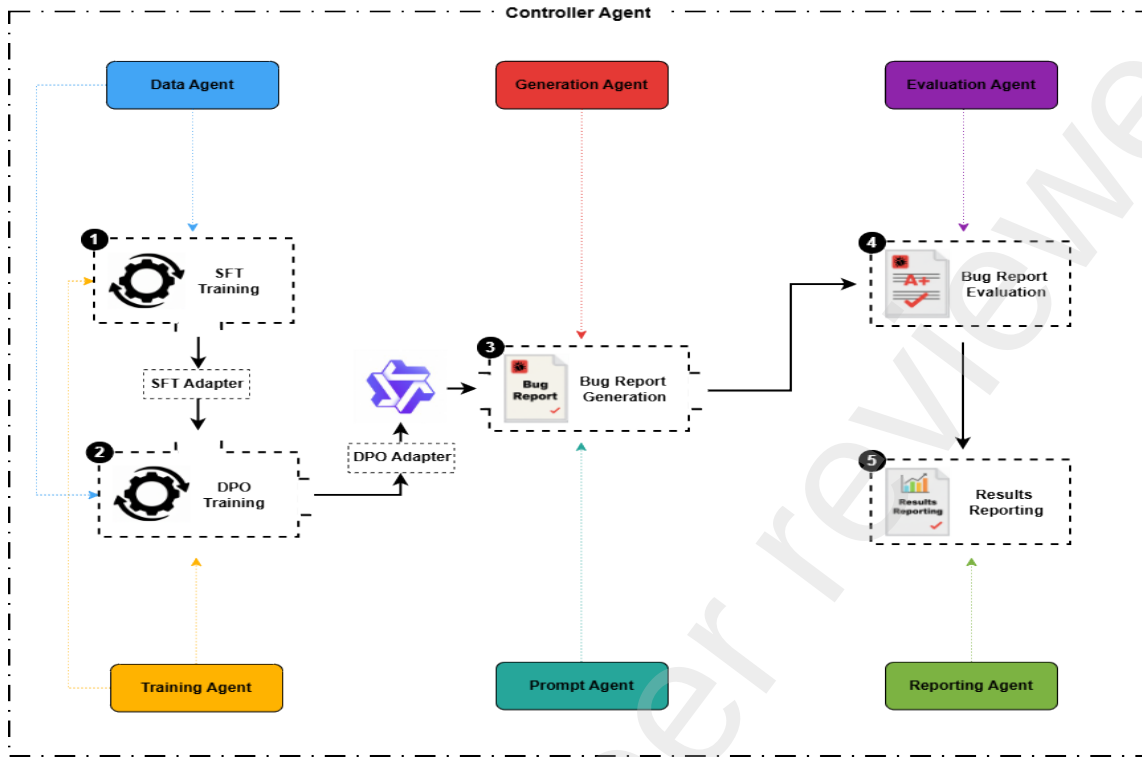


Figure 2. The overall architecture of the proposed multi-agent framework.

3.1. Data Preprocessing

This study used a Bugzilla based bug report dataset that was built in prior work [14]. Approximately 15,000 reports were collected in the initial stage, and only those explicitly marked as fixed or closed were selected for the candidate training set. Resolved reports generally contain more complete descriptions of reproduction steps, execution environments, expected results, and actual outcomes, which provide the structured information required for model training.

However, the raw dataset still included many incomplete reports and inconsistently formatted entries. Reports containing only a stack trace, partial code snippets, or missing descriptions of the environment or observed results could reduce training quality and therefore needed to be removed. To address these issues, a two phase data cleaning procedure was applied.

In the first phase, regular expressions were used to retain only the reports that contained all of the following elements: summary, steps to reproduce, expected result, actual result, and additional information. In the second phase, each report underwent CTQRS automated evaluation, and only those scoring fourteen points or higher were selected [16]. This threshold served as a criterion for identifying high quality reports with sufficient structural completeness and information depth.

Through this process, a total of 3,966 reports were obtained. To further verify dataset suitability, the researchers manually reviewed a sample of two hundred reports. The final dataset is accessible through the same source repository, ensuring fairness and transparency in data usage [14].

3.2. Data Agent

The Data Agent performs two essential data preparation functions within the proposed framework. The first function is splitting and managing the original data for SFT training [24] and RAG prompting [13]. The second function is constructing preference pairs for DPO training [8].

For the first function, the Data Agent loads the original Bugzilla dataset and partitions it into training, validation, and test sets using an 8:1:1 ratio [14]. A fixed random seed is applied to maintain experimental reproducibility. The resulting split files are cached and later used consistently during the SFT phase of the Training Agent and the RAG index construction of the Prompt Agent.

The second function focuses on generating DPO training data to enable fair comparison, which represents a key contribution of this study. This stage operates as a preparatory step for the DPO training flow managed by the Controller Agent. The Data Agent first loads the SFT baseline model trained by the Training Agent. It then processes each summary in the training dataset by inputting it into the SFT model to produce an “SFT generated report (Generated).”

Next, the Data Agent invokes the Evaluation Agent to assess the quality of both the original “golden report (Golden)” and the “SFT generated report (Generated).” The assessment relies on a composite score derived from CTQRS [16], ROUGE-1 Recall [17], and SBERT [18]. The Data

Agent compares the composite scores of the two reports and designates the higher-scoring one as “chosen” and the lower-scoring one as “rejected.” Report pairs are excluded from training when both reports fall below a minimum quality threshold or when the score difference is negligible, which helps prevent noise in the DPO dataset.

The Data Agent saves the resulting preference pairs in JSON format, and these pairs are directly used by the Training Agent during the DPO training stage. This module separates data preparation from DPO data creation, which enhances modularity and reproducibility throughout the entire pipeline.

3.3. Training Agent

The Training Agent serves as the central module that oversees the model training stage of the proposed framework. It sequentially performs two processes: Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO) [8]. Depending on requests from the Controller Agent, this module executes either SFT or DPO training and applies LoRA (Low-Rank Adaptation) with 16-bit precision to enable parameter-efficient tuning [24].

In the SFT stage, the module receives the training dataset prepared by the Data Agent and fine-tunes a pretrained language model (Qwen2.5–7B Instruct) [25] through supervised learning. This step establishes the baseline model, allowing it to learn the structural format of bug reports, logical flow, and stylistic consistency, which strengthens its domain-specific capabilities. The resulting parameters are stored as a LoRA adapter, forming the foundation for the subsequent DPO phase.

The DPO stage refines the model further by aligning its behavior based on preference signals. When this stage begins, the Training Agent merges the LoRA adapter produced during SFT with the base model to finalize the baseline. Using this merged model, it then trains a new LoRA adapter with the preference pair data constructed by the Data Agent. Instead of relying on absolute quality scores, DPO learns the relative preference ranking between chosen and rejected reports. Through this process, the model enhances the richness of content and the consistency of reasoning on top of the structural completeness established during SFT.

As a result, the Training Agent first enables the model to acquire the structural foundation of bug reports through SFT and then guides it toward relative quality alignment through DPO. This dual-stage workflow supports the generation of reports that reflect both formal robustness and coherent content. The module functions as the training core of the proposed framework, ensuring improved model performance and reproducible learning outcomes.

3.4. Prompt Agent

You are an expert QA engineer and explainability analyst.

Convert the user's unstructured input into a CTQRS-maximized bug report while using explainable evidence and step-by-step internal reasoning.

Structured Output

[Summary]

<one sentence: component/module + trigger condition + failure symptom>

[Steps to Reproduce]

1. <user action> - <immediate result/observation>

2. <user action> - <immediate result/observation>

3. <user action> - <immediate result/observation>

4. <optional extra user action> - <result>

[Expected Behavior]

<precise, testable outcome using the same key nouns as in Actual>

[Actual Behavior]

<what actually happens; include any error text or visible symptom; reuse the same key nouns>

[Environment]

- OS: <name version>

- Device/Browser: <model or browser name+version>

- App Version/Build: <semver or build tag>

- Network: <Wi-Fi/LTE/VPN/Proxy>

- Locale/Region: <e.g., en-US>

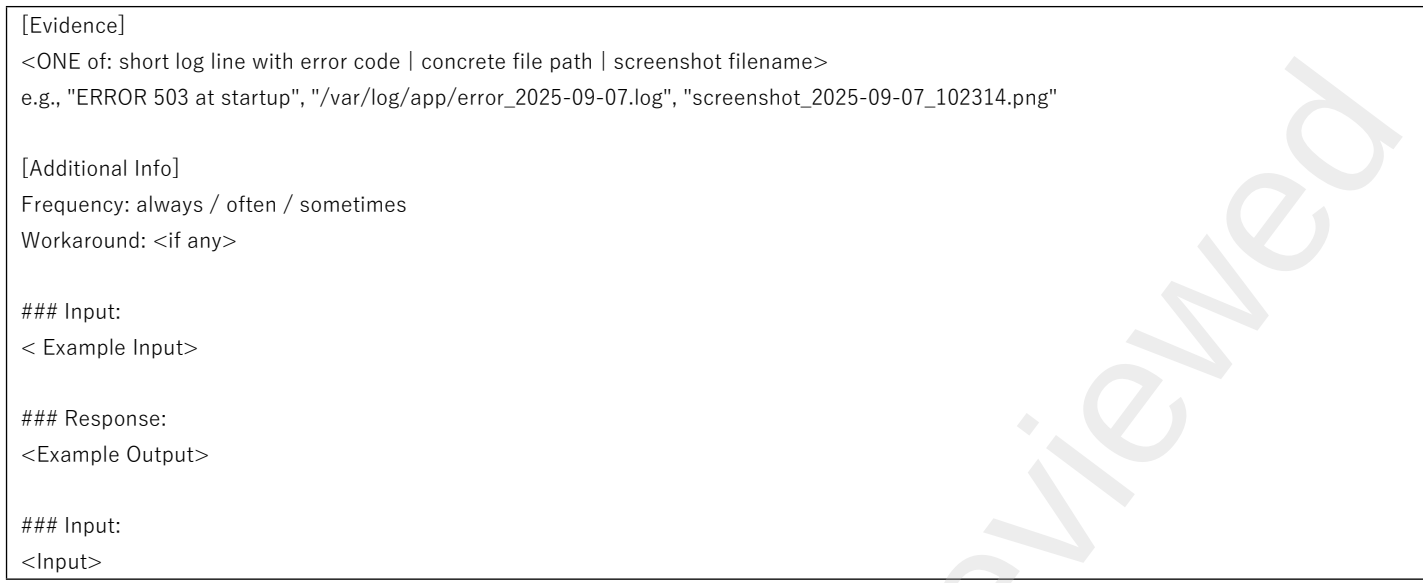


Figure 3. The prompt structure integrating the base system template and a 1-shot (k=1) RAG-based example.

The Prompt Agent is a core module responsible for constructing the final input prompt delivered to the LLM when the Controller Agent issues an inference request. This agent performs two primary functions: (1) applying a base template that enforces structural consistency, and (2) combining dynamically retrieved few-shot examples through retrieval-augmented generation (RAG) [13].

First, the Prompt Agent applies a predefined Base System Prompt to every input. This template assigns the model a clear role as an “expert QA engineer” and provides a structured format that includes predefined fields such as [Summary] and [Steps to Reproduce]. This design prevents structural instability and information omissions that may occur when the model produces free-form responses, which helps maintain consistency in the output. Second, depending on the experimental setting ($k > 0$), the agent performs RAG-based few-shot prompting. When the Controller Agent enables RAG, the Prompt Agent embeds all input summaries from the training dataset prepared by the Data Agent and builds a FAISS search index in advance [27]. Once a new input summary is provided, the agent retrieves the k most semantically similar (summary, reference report) pairs from the index in real time ($k=1$ or 2 in this study). The retrieved examples are inserted between the base template and the actual input as in-context learning samples, encouraging the model to refer to expression patterns from similar cases and generate more natural and contextually rich outputs. Figure 3 illustrates the structure of the combined prompt when $k=1$.

As a result, the Prompt Agent functions not merely as a pass-through component but as an input assembly module that integrates template-driven structural consistency with RAG-based contextual reference. This integration supports stable and high-quality report generation by the model.

3.5. *Generation Agent*

The Generation Agent is the primary execution module responsible for producing the final bug report after receiving an inference request from the Controller Agent. When instantiated by the Controller Agent, it loads the language model (Qwen2.5 7B Instruct) equipped with the SFT or DPO LoRA adapter produced by the Training Agent [24], [25]. It also incorporates the Prompt Agent and Evaluation Agent as internal modules, enabling an integrated pipeline for generation and evaluation.

The Generation Agent processes input data (summaries) in batches. For each batch, it invokes the Prompt Agent to construct a RAG-based prompt and then passes it to the model to generate the report. To ensure reproducibility during experimentation, the system applies a deterministic decoding policy (Temperature = 0, do_sample = False) based on the Controller Agent’s configuration. This policy guarantees identical outputs for identical inputs and models, minimizing variation caused by randomness and enabling fair comparison between the SFT and DPO approaches.

The generated reports are not passed through an external pipeline. Instead, they are immediately evaluated within the execution loop by the internal Evaluation Agent. The Evaluation Agent compares each generated report to its reference report and computes CTQRS, ROUGE-1 Recall, and SBERT metrics in real time [16]–[18].

The Generation Agent therefore operates as more than a simple output module. Under the direction of the Controller Agent, it performs prompt construction, model inference, and real-time evaluation in an integrated manner. This design provides a rigorous and reproducible procedure for assessing the proposed framework and serves as the central execution engine that ensures reliability across the entire system.

3.6. *Evaluation Agent*

The Evaluation Agent is a core utility module that quantifies report quality throughout the proposed framework. It is designed as an independent module so that it can be called and reused in both the preparatory stage for constructing DPO preference pairs and the inference stage for evaluating the final report. Centralizing the evaluation logic in a single agent ensures consistent assessment criteria and strengthens

modularity across the framework.

The Evaluation Agent receives a generated report and a reference report as input, and computes scores in three dimensions: structural completeness, lexical quality, and semantic consistency. First, structural completeness is assessed using the CTQRS metric [16]. The agent loads CTQRS scripts, including those used in baseline models, and evaluates whether the report includes key components such as reproduction steps, environment details, and expected and actual outcomes with coherent structure. Second, lexical quality is measured with the ROUGE-1 Recall metric [17], which focuses on how well the generated report preserves essential vocabulary from the reference report without omissions. Third, semantic consistency is evaluated using cosine similarity of SBERT (Sentence-BERT) embeddings [18]. This quantifies how closely the two reports align in meaning even when their surface wording differs, complementing ROUGE by capturing contextual coherence.

The Evaluation Agent automates these procedures, returns the three metric scores to the Controller Agent, and the Reporting Agent later aggregates and stores them. The Evaluation Agent does not communicate directly with the Reporting Agent, and this separation enhances the overall modular structure of the system.

3.7. Reporting Agent

The Reporting Agent is the final utility module responsible for aggregating and persistently storing all experimental results generated within the inference pipeline of the proposed framework. This agent does not operate independently; it is invoked only after the Controller Agent completes the entire inference workflow. This one-directional data flow separates responsibilities so that neither the Generation Agent nor the Evaluation Agent needs to handle result-storage logic, which strengthens the system’s modularity and maintainability.

Upon receiving a call from the Controller Agent, the Reporting Agent performs result reporting at two levels. First, it stores the results of individual runs. Whenever inference finishes for each model and prompt configuration, the Controller Agent triggers this function, and the Reporting Agent records the detailed scores for each generated bug report (CTQRS, ROUGE-1 Recall, SBERT) in an Excel file. Second, once the Controller Agent completes all experimental combinations, the Reporting Agent produces an integrated summary. After receiving the full set of experimental data from the Controller Agent, it computes the average performance for each configuration (CTQRS, ROUGE-1 Recall, SBERT) and stores these metrics in a separate summary Excel file.

As a result, the Reporting Agent functions as more than a storage component; it centrally manages the transformation of raw measurements from the Evaluation Agent into researcher-interpretable outputs. This design supports the core principles of reproducibility and modularity that underlie the contributions of this study at the code level.

3.8. Controller Agent

Controller Agent is the central orchestration module responsible for managing the entire process of the proposed framework. It ensures that all individual agents operate independently while still functioning as a coherent and unified workflow. This architectural design is essential for maintaining the reproducibility and modularity that constitute key contributions of this study.

A primary responsibility of the Controller Agent is to control the system’s execution mode based on command-line arguments. Users can explicitly specify training modes (SFT or DPO) or the inference mode through flags, enabling them to switch and reproduce experimental settings with ease while using the same codebase.

When the system is set to training mode, the Controller Agent invokes the Data Agent to perform dataset partitioning and load training data, then triggers the Training Agent to execute the SFT or DPO training pipeline. During this process, the Controller Agent manages the path of the latest SFT or DPO adapters to maintain training continuity.

In inference mode, the Controller Agent creates instances of the Prompt Agent and the Evaluation Agent. It then injects these two agents along with the trained LoRA adapter into the Generation Agent, delegating the inference procedure. After the Generation Agent completes prompt construction, model inference, and real-time evaluation using CTQRS, ROUGE-1 Recall, and SBERT, it returns a final list of results. The Controller Agent passes this output to the Reporting Agent, which consolidates and stores the data.

Overall, the Controller Agent functions as more than a task scheduler. It centrally coordinates every component of the pipeline and maintains system-wide reproducibility and consistency. Each subordinate agent intentionally operates with restricted autonomy, performing only predefined tasks under the explicit control of the Controller Agent. This centralized control structure prevents nondeterministic behavior and supports reliable experimentation.

4. Experiment

4.1. Experiment Settings

The experimental evaluation was conducted to assess the performance of the proposed framework. All experiments were carried out using the Qwen2.5-7B-Instruct model [25] and implemented with the Hugging Face Transformers [28], TRL [29], and PEFT [30] libraries. To maintain the principle of a fair comparison, both the baseline model (SFT) and the proposed model (DPO) were fine-tuned using the LoRA method in 16-bit precision [24].

The hyperparameters for training the baseline (SFT) model were as follows: batch size of 1, 8 gradient accumulation steps, a learning rate of $2e-4$, LoRA rank (r) of 16, LoRA alpha of 32, LoRA dropout of 0.05, a maximum sequence length of 2048, and a total of 3 epochs. The random seed

was fixed at 42.

Training of the proposed model (DPO) was performed using the SFT-trained model as the starting point, and the default settings of the DPOConfig in the TRL library were applied [29]. Fine-tuning was conducted with LoRA (16-bit) for one additional epoch.

For the dataset, the experiments utilized 3,966 summary-ground truth pairs derived from Bugzilla and released in prior work [14], [15]. The Data Agent split the dataset into training, validation, and test sets at an 8:1:1 ratio using a fixed seed of 42. Preference pairs for DPO were constructed using the procedure described in Section 3.1, excluding pairs with an overall score below 0.5 or a score difference smaller than 0.01.

When constructing the RAG (few-shot) prompts, the Prompt Agent retrieved $k=1$ or $k=2$ similar examples exclusively from the training set using the BAAI/bge-large-en-v1.5 embedding model [31] and a FAISS index [27], which prevents any form of data leakage. The Generation Agent employed deterministic decoding (Temperature=0, do_sample=False) to ensure reproducible outputs.

All experiments were executed in an Ubuntu 24.04 (WSL2) environment using a single NVIDIA RTX 4090 GPU. The primary software stack consisted of PyTorch 2.5.1 [32], Transformers 4.57.1 [28], TRL 0.23.0 [29], and PEFT 0.17.1 [30].

4.2. Dataset

This study uses the 3,966 summary-ground truth pairs from the Bugzilla-based dataset released as the baseline in the prior work. We directly adopted the official version provided in the original authors' GitHub repository, and no additional cleaning or preprocessing was performed. This choice ensures consistent data quality and enables a fair and fully reproducible comparison with previous research.

The original dataset was constructed by collecting approximately 15,000 reports from Bugzilla, an open source bug tracking system, and then selecting only those whose status was labeled as "fixed" or "closed." Each report contains a paired structure consisting of a summary and its corresponding ground truth report. The dataset had already undergone cleaning and validation in the prior study, and all incomplete reports had been removed. Furthermore, the dataset was filtered using the CTQRS metric, and only reports with a CTQRS score of at least 14 were included.

For this work, we used the refined set of 3,966 summary-ground truth pairs exactly as released. The data were split into training, validation, and test sets at a ratio of 8:1:1, applying a fixed random seed to ensure identical partitions across experiments. This process preserves the statistical distribution of the data while supporting fairness and reproducibility in the experimental results.

4.3. Evaluation Metrics

This study evaluates the quality of generated bug reports from three perspectives: structural completeness, lexical coverage, and semantic consistency. The definitions of each metric and the criteria used for interpretation are summarized in Table 1.

Metric	Evaluated Aspect	Description
CTQRS [16]	Structural Completeness	The evaluation examines how faithfully a bug report satisfies five criteria: atomicity, conciseness, completeness, understandability, and reproducibility. Each criterion is scored on a scale of up to 17 points and then normalized to a range of 0 to 1. A score below 0.5 indicates a low-quality report with substantial information gaps, while a score of 0.9 or higher represents a structurally complete report. This study adopts the automated CTQRS evaluation script released in prior work without modification.
ROUGE-1 Recall [17]	Lexical Coverage	The evaluation measures how well a generated report captures the key terms and essential phrasing of the ground-truth report. Scores in the range of 0.3 to 0.5 indicate substantial omission of important information, while scores of 0.8 or higher represent adequate lexical coverage. Recall serves as a primary indicator for determining whether the main content has been reproduced without information loss.
SBERT [18]	Semantic Consistency	The evaluation computes semantic similarity between the generated report and the ground-truth report using cosine similarity of Sentence-BERT embeddings. A score below 0.6 indicates semantic drift, while a score of 0.8 or higher suggests that the generated report preserves the intended meaning even when the wording differs.

Table 1. Evaluation metrics used for assessing bug report quality.

The evaluation framework follows the existing baseline that relies on CTQRS, ROUGE-1 Recall, and SBERT, allowing an integrated assessment of structural, lexical, and semantic quality. This approach enables a multidimensional analysis that verifies whether the generated reports go beyond reproducing key terms and actually maintain structural completeness, broad lexical coverage, and semantic consistency. All metrics are presented as percentages, where higher values indicate better quality, which provides an intuitive and uniform basis for comparing model performance. The automated evaluation process enhances objectivity and reproducibility, and future work will incorporate human evaluation by

developers to further examine readability, practical usefulness, and the effort required to reproduce outcomes.

4.4. Baseline and Research Questions

This study establishes four research questions to examine the absolute performance of the proposed training strategy, the enhancement effect of auxiliary techniques, model-type differences, and the comparative suitability of our approach against existing methods. This structure is designed to isolate the core contribution of the proposed model based on DPO training and to clarify how additional techniques, such as few-shot prompting, influence overall performance.

The research questions are defined as follows.

- RQ1. Verification of the absolute performance of the proposed DPO model
The first research question focuses on determining whether the proposed model achieves a stable and convincing level of quality when evaluated independently. The study applies DPO (Direct Preference Optimization) training on top of a standard SFT (Supervised Fine-Tuning) stage to strengthen the model and then verifies whether the DPO model alone shows consistent performance across the CTQRS, ROUGE-1 Recall, and SBERT metrics. This assessment evaluates whether the training strategy itself offers sufficient effectiveness.
- RQ2. Evaluating the enhancement effect of adding few-shot prompting to DPO
The second research question examines how few-shot prompting (1-shot and 2-shot) modifies the performance of the proposed model. While few-shot prompting is not the core contribution of this work, it is a supportive strategy that can promote additional improvements through contextual augmentation. By comparing the DPO model with the DPO plus few-shot configurations, the study quantifies the degree to which few-shot prompting enhances structural completeness, lexical recall, and semantic consistency.
- RQ3. Comparing the influence of few-shot prompting on Baseline vs. DPO
The third research question analyzes how few-shot prompting behaves under different training paradigms. The same prompt structure and the same 1-shot and 2-shot settings are applied to both the Baseline (SFT) and DPO models, and the study evaluates the relative differences in their responses. This comparison clarifies the extent to which few-shot prompting compensates for the limitations of the Baseline model and what additional benefits arise when it is combined with DPO.
- RQ4. Assessing the comparative suitability of the proposed method against the Baseline and GPT-4o
The fourth research question evaluates whether the proposed model serves as a more appropriate alternative to existing approaches and external high-performance language models. The comparison includes Baseline (SFT), Baseline with few-shot prompting, the proposed DPO model, the proposed DPO with few-shot prompting, and GPT-4o under 0-shot and 3-shot conditions. Using the same dataset and identical evaluation protocol, the study examines relative performance differences in terms of CTQRS, ROUGE-1 Recall, and SBERT. This assessment determines whether the proposed approach remains competitive not only against SFT-based models but also against large external models.

Together, RQ1 through RQ4 establish a coherent framework for examining the effectiveness of the proposed training method from multiple angles. Each question is designed to investigate whether the model performs sufficiently on its own, whether the auxiliary technique of few-shot prompting provides additional benefits, how the same technique behaves across different training schemes, and whether the proposed approach offers advantages over existing models or external large-scale systems. Through this step-wise investigation, the study aims to show that the DPO-based bug report generation method provides not only quantitative improvements but also more reliable gains in structural completeness, lexical recall, and semantic coherence compared with earlier approaches.

4.5. Experimental Results

This section presents the experiments conducted for the four research questions (RQs). The results are organized according to each question's objective. Figure 4 reports the absolute performance of the DPO model, Figure 5 presents the configuration comparison between DPO and few-shot prompting, Table 2 summarizes the shot-condition differences between the Baseline and DPO models, and Figure 6 provides the comparative results among the Baseline, DPO, and GPT-4o models.

The results in Figure 4 show that the DPO model achieved 81.1 percent on CTQRS, 73.2 percent on ROUGE-1 Recall, and 85.0 percent on SBERT, indicating strong and stable absolute performance across all three metrics. The consistently high CTQRS and SBERT scores indicate that the model maintains sufficient structural completeness and semantic coherence, while the Recall score confirms that the model effectively reproduces key expressions and essential information, reflecting solid lexical fidelity.



Figure 4. Absolute performance of the DPO model (0-shot) across the three evaluation metrics.

The additional 5,000-iteration percentile bootstrap analysis also revealed a stable pattern. For CTQRS, the mean was approximately 81 percent, with a confidence interval ranging from 79.96 percent to 82.26 percent. ROUGE-1 Recall showed a mean of roughly 73 percent, distributed between 72.16 percent and 74.33 percent. SBERT formed a similar pattern, with an average of about 85 percent and an interval from 84.24 percent to 85.78 percent. These findings indicate that the DPO model maintains consistent behavior even when evaluated through repeated resampling rather than a single experimental condition, reinforcing the model's overall stability.

RQ2 examines how few-shot prompting affects the performance of the DPO model. As shown in Figure 5, CTQRS increased from 81.1 percent for the original DPO model to 83.2 percent for DPO (1-shot) and 82.9 percent for DPO (2-shot). This trend confirms that few-shot prompting has a positive influence on structuring the generated reports.

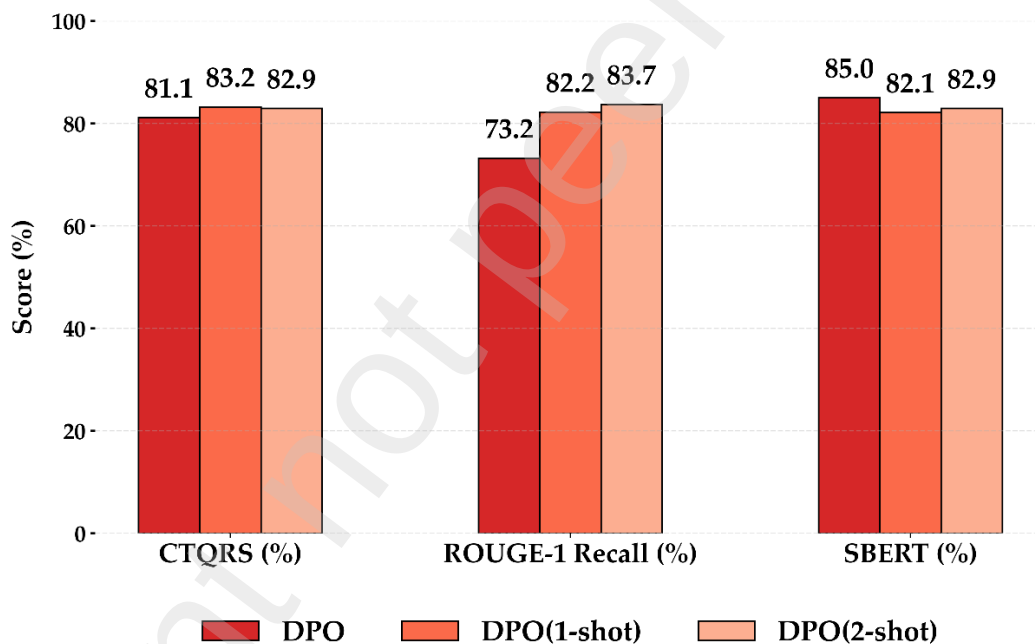


Figure 5. Performance comparison of the DPO model with 0-shot, 1-shot, and 2-shot RAG prompting.

ROUGE-1 Recall increased substantially, rising from 73.2 percent with the DPO model to 82.2 percent for DPO (1-shot) and 83.7 percent for DPO (2-shot). This indicates that the number of shots has a clear influence on lexical fidelity. The results suggest that few-shot prompting supplements the diversity and reproducibility of key information elements at a high level.

In contrast, SBERT decreased slightly, moving from 85.0 percent for the DPO model to 82.1 percent for DPO (1-shot) and 82.9 percent for DPO (2-shot). Given that the magnitude of the decrease is small, this shift is interpreted not as a reduction in semantic quality but rather as an effect of altered sentence composition or phrasing introduced during the few-shot prompting process.

From a balanced-performance perspective, DPO (1-shot) maintains stable behavior across all three metrics without large deviations. This indicates that it provides the most balanced configuration when applying few-shot prompting in its initial stage.

RQ3 investigates how the same few-shot prompting setup affects the Baseline (SFT) and DPO models differently, focusing on metric-level changes. As shown in Table 2, the Baseline model's CTQRS increased from 77.0 percent to 79.9 percent with one shot, then declined to 77.6 percent with two shots. By contrast, the DPO model moved from 81.1 percent to 83.2 percent (1-shot) and 82.9 percent (2-shot), remaining within a narrower and more consistent range. In other words, the Baseline model exhibited an upward shift followed by a larger downward fluctuation

depending on the shot configuration, while the DPO model maintained a comparatively stable level in both conditions. Based solely on the numerical trends, the DPO model retains a more consistent structural quality when few-shot prompting is applied.

Configuration	CTQRS(%)	ROUGE-1 Recall(%)	SBERT(%)
Baseline	77	61	85
Baseline(1-shot)	79.9	82.1	84.4
Baseline(2-shot)	77.6	80.1	83.3
DPO	81.1	73.2	85
DPO(1-shot)	83.2	82.2	82.1
DPO(2-shot)	82.9	83.7	82.9

Table 2. Comparative effects of 1-shot and 2-shot prompting on the Baseline (SFT) and DPO models across all metrics.

The pattern of change in ROUGE-1 Recall also differed by model. The Baseline model fluctuated from 61.0 percent to 82.1 percent with one shot and then decreased to 80.1 percent with two shots, indicating unstable behavior. In contrast, the DPO model increased from 73.2 percent to 82.2 percent (1-shot) and 83.7 percent (2-shot), showing a gradual and consistent upward trend. These results indicate that the Baseline model is more sensitive to the shot configuration, while the DPO model improves in a steadier manner.

For the SBERT metric, the Baseline model decreased from 85.0 percent to 84.4 percent (1-shot) and 83.3 percent (2-shot), and the DPO model similarly declined from 85.0 percent to 82.1 percent (1-shot) and 82.9 percent (2-shot). Because both models show small declines in the same direction, it can be inferred that applying few-shot prompting may introduce variability in semantic similarity-based metrics. The implications of these changes and the potential trade-offs are examined separately in Section 5.1.

RQ4 compares the relative performance of the Baseline, the DPO model, DPO (1-shot), GPT-4o (0-shot), and GPT-4o (3-shot). In the CTQRS results shown in Figure 6, DPO (1-shot) achieved the highest score at 83.2 percent. The other configurations yielded the following values: 81.1 percent for the DPO model, 77.0 percent for the Baseline, and 73.0 percent and 75.0 percent for GPT-4o with zero shot and three shots, respectively. This distribution indicates that DPO (1-shot) produces the most structurally complete reports, and the DPO model also maintains relatively high structural completeness.

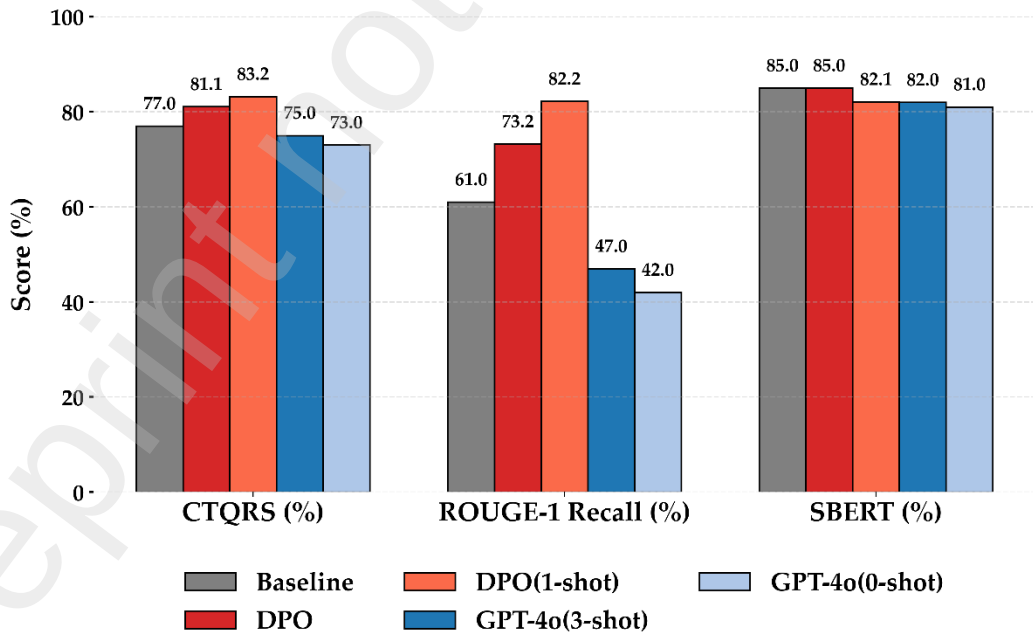


Figure 6. Performance comparison of the proposed DPO models against the SFT Baseline and GPT-4o (0-shot and 3-shot) configurations.

For the ROUGE-1 Recall metric, DPO (1-shot) again achieved the highest value at 82.2 percent. This was followed by the DPO model at 73.2 percent, the Baseline at 61.0 percent, and GPT-4o at 42.0 percent (0-shot) and 47.0 percent (3-shot). These results indicate that DPO (1-shot)

captures the broadest range of key expressions and information elements, while the DPO model also maintains comparatively strong reproducibility.

In the SBERT metric, both the Baseline and the DPO model recorded the highest value at 85.0 percent. DPO (1-shot) followed at 82.1 percent, and GPT-4o recorded 81.0 percent (0-shot) and 82.0 percent (3-shot). This pattern suggests that the Baseline and DPO models produce bug reports with strong semantic coherence.

Taken together, DPO shows balanced performance across all three metrics and consistently produces high-quality outputs in terms of structural completeness, lexical coverage, and semantic consistency. Notably, DPO (1-shot) generates reports with strengthened structural completeness and lexical breadth, providing numerical evidence that few-shot prompting can meaningfully enhance the generation quality of DPO-based models.

5. Discussion

5.1. Analysis of Experimental Results

The experimental findings indicate that the proposed framework based on Direct Preference Optimization (DPO) consistently outperforms the Supervised Fine Tuning (SFT) baseline across all evaluation metrics, including structural completeness (CTQRS), lexical recall (ROUGE-1 Recall), and semantic consistency (SBERT). The DPO model shows clear advantages over the baseline in CTQRS and SBERT because SFT mainly imitates patterns present in gold data, whereas DPO learns the distinction between preferred reports and less favorable ones. This enables the model to internalize the logical structure and narrative style favored by human annotators. The strong performance in CTQRS shows that DPO effectively captures the essential elements of a bug report, including reproduction steps and environmental details, without omissions.

The behavior observed when applying few-shot prompting also reveals notable characteristics. Providing one-shot or two-shot examples to the DPO model led to substantial improvements in CTQRS and ROUGE-1 Recall, while SBERT exhibited a slight decrease, showing a trade-off between lexical specificity and semantic similarity. For instance, DPO with one-shot prompting increased ROUGE-1 Recall by approximately nine percentage points compared to zero-shot, reaching 82.2 percent, while SBERT decreased from 85.0 percent to 82.1 percent. A similar pattern was observed in the baseline model, suggesting that retrieved examples introduced through Retrieval Augmented Generation (RAG) exert combined influence on the outputs.

The discrepancy between metrics can be explained by two primary factors: increased lexical specificity and shifts in writing style. First, the rise in ROUGE-1 Recall indicates that few-shot examples expose the model to domain-specific terminology and contextual cues, allowing it to better capture the key tokens present in the ground truth reports. This suggests that retrieved examples encouraged the model to produce more detailed content without omission. Second, the slight decline in SBERT appears to stem not from semantic errors but from stylistic drift caused by the model adopting the phrasing and structure of the few-shot examples. Even when the generated report conveys the same information, variations in sentence length or structural patterns can reduce cosine similarity in embedding space.

Despite the modest reduction in SBERT, the significant gains in CTQRS and ROUGE-1 Recall indicate that few-shot prompting enhances the richness of information and structural completeness rather than degrading report quality. Combining DPO training with one-shot prompting offers the most balanced performance, securing strong structural stability and lexical accuracy while maintaining reliable semantic alignment.

5.2. Threats to Validity

This study assessed an automated bug report generation model by integrating Direct Preference Optimization (DPO) training with Retrieval-Augmented Generation (RAG) within a multi-agent pipeline. While this integrated design provides experimental consistency and extensibility, several factors may pose threats to the validity of the experimental design and the interpretation of the results. This section examines potential threats and corresponding mitigation strategies across internal, external, construct, and conclusion validity.

Internal validity concerns whether the observed results can be attributed to the manipulation of experimental variables rather than external factors. To maintain fairness and reproducibility, 3,966 Bugzilla reports were partitioned into an 8:1:1 ratio using a fixed random seed. This setup ensured consistent data splits across comparative experiments and prevented data leakage; however, reliance on a single seed and static partitioning limits the ability to assess model robustness to shifts in data distribution or random variation. Deterministic decoding (temperature = 0, do_sample = False) was used to remove variability in generation, yet this configuration does not account for performance differences under diverse sampling strategies that may be required in deployment scenarios. Future work should incorporate multiple seeds, repeated data splits, and evaluations under varied decoding configurations to strengthen internal validity.

External validity addresses whether the proposed approach can generalize to other environments or domains. The experiments in this study were conducted exclusively on the Bugzilla dataset, a well-known open-source issue tracking platform. Commercial systems such as GitHub Issues and Jira differ substantially in metadata design, access policies, and user writing styles. Domain-specific contexts such as mobile applications, large-scale enterprise systems, or security vulnerability reporting can also exhibit distinct reporting patterns. Although the modular multi-agent design in this study supports cross-platform portability, cross-platform evaluations have not yet been completed. Ongoing preliminary assessments on additional datasets and future replication studies across diverse ecosystems will be necessary to show the generalizability of the proposed framework.

Construct validity concerns whether the evaluation metrics accurately reflect the properties the study aims to measure. The primary metrics

used were CTQRS for structural completeness, ROUGE-1 Recall for lexical coverage, and SBERT for semantic consistency. CTQRS measures structural fidelity but does not capture fluency or stylistic variation, and ROUGE has well-known limitations in assessing contextual appropriateness. SBERT measures semantic similarity, yet results may vary depending on the characteristics of the underlying embedding model. Furthermore, automated metrics may not fully represent developers' perceptions of readability or practical usefulness. Future research will introduce length normalization penalties to reduce over-generation and incorporate human evaluations to assess the practical value of the generated reports.

Conclusion validity involves the accuracy of statistical inferences. Due to the absence of instance-level outputs for the baseline model, direct significance testing such as paired t-tests could not be performed. To address this limitation, the study applied a percentile bootstrap approach with 5,000 resampling iterations on 397 test samples to estimate 95 percent confidence intervals. The resulting intervals were narrow, and the lower bounds for the proposed model were higher than those of the baseline, indicating that the observed performance gains were unlikely to be due to random error. Further refinement of baseline categories in future work will enable more precise statistical testing and improve the strength of the conclusions.

Overall, the proposed framework showed consistent performance improvements over the baseline. Nevertheless, factors such as reliance on a single seed, platform-specific bias, and the limitations of automated metrics remain challenges that must be addressed. These threats can be mitigated through multi-dataset validation, the adoption of hybrid evaluation schemes, and pilot studies in real operational environments, which will provide a stronger foundation for showing the practicality and reliability of the proposed approach.

6. Related Work

Research on automating and improving the quality of bug reports has evolved widely, ranging from traditional representation learning to the recent adoption of large language models (LLMs). Early work focused on learning effective representations of bug report features. For instance, Fang et al. [33] introduced a universal learning representation for bug reports to support various downstream tasks. Although such approaches were useful for classification or priority prediction, they had inherent limitations when directly generating reports or ensuring structural completeness.

To address these limitations, researchers began incorporating LLMs, initially relying on supervised fine-tuning (SFT). Studies like CodeAlpaca [34] explored the potential of tuning LLMs for code-related tasks through instruction-following datasets. However, Gudibande et al. [35] argued that SFT alone, which is essentially imitation learning, often fails to internalize the desired quality or human preferences, calling this a "false promise." Their findings highlighted the need for training strategies that align model outputs with human preference signals. This motivation guided the development of methods such as RLHF [36], and more recently Direct Preference Optimization (DPO) [8], which simplifies the RLHF pipeline by directly optimizing preference learning. DPO represents the approach adopted in this study.

Efforts to apply LLMs to software engineering (SE) tasks have also grown steadily. Jiang et al. [37] and Sobania et al. [38] examined how LLMs influence practical SE tasks such as automated program repair and bug fixing. In a direction similar to the retrieval-augmented generation (RAG) approach used in this study [13], Shrivastava et al. [39] proposed dynamically constructing prompts through the retrieval of large repository-level contexts. Their work represents an important precedent for applying retrieval-based context augmentation to SE problems. Fakhoury et al. [40] additionally investigated modifying code from natural-language issue descriptions, addressing a problem domain closely related to ours. Building upon this thread of research, the present study adopts the Qwen model architecture [41] as its base LLM.

Research on self-verification and agent-based pipelines for improving LLM output quality has provided direct theoretical foundations for our multi-agent design. Madaan et al. [42] introduced the concept of Self-Refine, where a model iteratively provides feedback and enhances its own output. Gou et al. [43] further developed the CRITIC framework, enabling LLMs to interact with external tools such as evaluation metrics to critique and revise their responses. This idea parallels the role of the Evaluation Agent in our system, which verifies output quality using automated metrics including CTQRS [16] and SBERT [18]. In addition, Dong et al. [44] applied a self-collaboration paradigm to code generation, reinforcing the validity of multi-agent collaboration strategies similar to those used in our architecture.

Research on evaluation metrics has also advanced to support reliable assessment of generated content. Beyond traditional metrics, Zhou et al. [45] introduced CodeBERTScore, a BERT-based semantic similarity metric adapted for the code domain. Their work aligns with our use of SBERT [18], alongside CTQRS [16] and ROUGE [17], to assess semantic and structural consistency. Within our framework, the Evaluation Agent acts not as an LLM evaluator but as a utility component that consistently executes automated metric scripts, ensuring evaluation reproducibility throughout DPO dataset construction and final performance assessment.

Although outside the core scope of this study, automated program repair (APR) research remains complementary to bug report generation. Silva et al. [46] applied LoRA-based adaptation to LLaMA for program repair in the RepairLLaMA system. Their approach shares methodological similarities with this study, which also uses LoRA [24] for preference-based DPO training. High-quality bug report generation can support downstream APR stages by providing richer diagnostic information.

Taken together, prior studies have contributed substantially to representation learning [33], SFT research and its limitations [34], [35], the application of retrieval-based context augmentation to SE tasks [39], and agent-driven self-verification approaches [42], [43]. Yet few attempts have combined modern preference-based training such as DPO [8] with RAG-based contextual enhancement [13] and implemented the entire workflow using a deterministic multi-agent architecture designed for experimental reproducibility and modularity. To bridge this gap, the present study integrates DPO, RAG, and a centralized controller-driven multi-agent framework to holistically enhance the structural, lexical, and semantic quality of LLM-generated bug reports.

7. Conclusion

This study proposes a multi-agent framework that integrates Direct Preference Optimization (DPO) training and Retrieval-Augmented Generation (RAG) to address the issue of unstructured and incomplete bug reports in open-source software environments. The system separates data processing, prompt construction, training, generation, evaluation, and reporting into independent agents, which allows the pipeline to maintain efficient coordination among modules. This architecture mitigates the scalability limitations of single-model approaches and achieves improved maintainability and reproducibility.

Experiments conducted on 3,966 summary-ground truth pairs from the Bugzilla dataset show that the proposed DPO-RAG framework delivers significant performance gains over the baseline supervised fine-tuning (SFT) model and GPT-4o. In particular, the DPO (0-shot) model achieves 81.1 percent on CTQRS, 73.2 percent on ROUGE-1 Recall, and 85.0 percent on SBERT. These results indicate that the generated reports are structurally strong (CTQRS 81.1 percent), faithfully reproduce key lexical elements of the reference reports (ROUGE-1 Recall 73.2 percent), and maintain high semantic consistency (SBERT 85.0 percent). Compared with the baseline SFT model (CTQRS 77.0 percent, ROUGE-1 Recall 61.0 percent, SBERT 85.0 percent), the proposed model improves CTQRS by 4.1 percentage points, suggesting a substantial reduction in omissions of essential components such as reproduction steps and environment details. The 12.2-point improvement in ROUGE-1 Recall confirms that the model captures important vocabulary more effectively, while the stable SBERT score shows that this structural and lexical enhancement does not compromise semantic fidelity. The framework also outperforms GPT-4o. DPO training strengthens structural completeness and logical coherence by learning from comparisons between model-generated and gold-standard reports, while the RAG-based prompt retrieves similar examples to reinforce contextual alignment and informational accuracy.

These findings provide two important implications. First, improvements in CTQRS indicate that missing elements such as reproduction procedures, execution environments, and expected versus actual results are reduced, which enhances the practical reproducibility of defect reports. Second, the combination of high ROUGE-1 Recall and stable SBERT scores suggests that the model captures essential terminology while preserving semantic coherence.

The main contribution of this work lies in presenting a multi-agent architecture that integrates preference-based training with retrieval-based contextual augmentation for bug report automation. Unlike prior approaches that rely primarily on single-stage fine-tuning, the proposed framework modularizes the entire process from data preparation to evaluation in a centrally orchestrated structure, providing evidence that LLM-based software maintenance automation is feasible in practice.

Several limitations and future directions remain. The experiments use a single random seed, fixed hyperparameters, and data restricted to the Bugzilla platform. To validate the generalizability of the method, future work should incorporate multi-seed experiments and bootstrap resampling for robustness, along with cross-platform evaluation on systems such as Jira and GitHub Issues, which differ in reporting style and domain characteristics.

Because automated metrics alone cannot fully capture the practical usefulness of bug reports, a qualitative evaluation with software developers is planned. This controlled user study will assess clarity, perceived usefulness, and time required to reproduce the defect. These results will be used to analyze the relationship between automated metrics and human judgment.

To accelerate real-world adoption, pilot studies will also be conducted within operational issue-tracking systems such as Jira and GitHub Issues. These studies aim to evaluate integration effort, user acceptance, and reliability under actual workflows, while tracking operational metrics such as average reporting time, reproduction success rate, and developer satisfaction. Successful validation would support the development of the framework as a plugin or API module interoperable with existing systems.

To ensure statistical reliability of the reported performance improvements, 5,000 rounds of percentile bootstrap resampling were performed to estimate the 95 percent confidence intervals for all key metrics. A limitation is that paired t-tests could not be conducted because instance-level outputs of the baseline model were not available. For full reproducibility, all configurations and prompt templates used for training and inference will be released through a public GitHub repository.

Future research should extend the framework along two major directions: generalization and practical deployment. First, cross-platform evaluation on Jira and GitHub Issues and multi-seed experiments are needed to confirm robustness. Second, to complement the limits of automated metrics, both controlled user studies and pilot deployments in real operational environments should be carried out. During these evaluations, operational metrics such as perceived usefulness, reproduction time, and developer satisfaction will be measured to determine the practical value the method can contribute to software maintenance workflows.

References

- [1] N. Bettenburg et al., "What makes a good bug report?" in *Proc. 16th ACM SIGSOFT Int. Symp. Foundations of Softw. Eng. (FSE-16)*, 2008, pp. 308–318.
- [2] Y. Fan, X. Xia, D. Lo, and A. E. Hassan, "Chaff from the wheat: Characterizing and determining valid bug reports," *IEEE Trans. Softw. Eng.*, vol. 46, no. 5, pp. 495–525, 2018.
- [3] S. Rastkar, G. C. Murphy, and G. Murray, "Automatic summarization of bug reports," *IEEE Trans. Softw. Eng.*, vol. 40, no. 4, pp. 366–380, 2014.
- [4] Atlassian. *Jira Software*. [Online]. Available: <https://www.atlassian.com/software/jira> (accessed Nov. 15, 2025).
- [5] T. Brown et al., "Language models are few-shot learners," in *Proc. Advances in Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 1877–1901.
- [6] OpenAI, "GPT-4 technical report," arXiv preprint arXiv:2303.08774, 2023.
- [7] A. Acharya and R. Ginde, "RAG-based bug report generation with large language models," in *Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, 2025.
- [8] R. Rafailov et al., "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," in *Proc. Advances in Neural Inf. Process. Syst. 36 (NeurIPS)*, 2023.
- [9] L. Wang et al., "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, Art. no. 186345, 2024.
- [10] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
- [11] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv preprint arXiv:2308.08155, 2023.
- [12] H. Chase, *LangChain*. GitHub repository, 2022. [Online]. Available: <https://github.com/langchain-ai/langchain> (accessed Nov. 15, 2025).
- [13] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. Advances in Neural Inf. Process. Syst. (NeurIPS)*, 2020.
- [14] GitHub, "Bug Report Summarization Benchmark Dataset," 2025. [Online]. Available: https://github.com/GindeLab/Ease_2025_AI_model (accessed Sep. 21, 2025).
- [15] Mozilla. *Bugzilla*. [Online]. Available: <https://www.bugzilla.org/> (accessed Nov. 15, 2025).
- [16] H. Zhang, Y. Zhao, S. Yu, and Z. Chen, "Automated quality assessment for crowdsourced test reports based on dependency parsing," in *Proc. 9th Int. Conf. Dependable Syst. and Their Appl. (DSA)*, 2022, pp. 34–41.
- [17] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proc. Workshop on Text Summarization Branches Out (ACL)*, 2004, pp. 74–81.
- [18] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Empirical Methods in Natural Lang. Process. (EMNLP)*, 2019, pp. 3980–3990.
- [19] C. Sun, D. Lo, and X. Khoo, "Towards more accurate retrieval of duplicate bug reports," in *Proc. ACM Int. Conf. Syst. and Softw. Eng. (ASE)*, 2011, pp. 253–262.
- [20] Z. He, A. Marcus, and D. Poshvanyk, "Using information retrieval and NLP to classify bug reports," in *Proc. Int. Conf. Program Comprehension (ICPC)*, 2010, pp. 148–157.
- [21] A. Lamkanfi et al., "Predicting the severity of a reported bug," in *Proc. IEEE Int. Conf. Mining Softw. Repositories (MSR)*, 2010, pp. 1–10.
- [22] H. Liu et al., "BugSum: Deep context understanding for bug report summarization," in *Proc. IEEE/ACM Int. Conf. Program Comprehension (ICPC)*, 2020, pp. 94–105.
- [23] Y. Shao and B. Xiang, "Towards effective bug report summarization by domain-specific representation learning," *IEEE Access*, vol. 12, pp. 37653–37662, 2024.
- [24] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
- [25] Qwen Team, "Qwen2.5 technical report," arXiv preprint arXiv:2409.12121, 2024.
- [26] A. Q. Jiang et al., "Mistral 7B," arXiv preprint arXiv:2310.06825, 2023.
- [27] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

- [28] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.
- [29] L. von Werra et al., *TRL: Transformer Reinforcement Learning*. GitHub repository, 2020. [Online]. Available: <https://github.com/huggingface/trl> (accessed Nov. 15, 2025).
- [30] S. Mangrulkar et al., *PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods*. GitHub repository, 2022. [Online]. Available: <https://github.com/huggingface/peft> (accessed Nov. 15, 2025).
- [31] Beijing Academy of Artificial Intelligence (BAAI), "BAAI BAAI/bge-large-en-v1.5 (English) Embedding Model," 2023. [Online]. Available: <https://huggingface.co/BAAI/bge-large-en-v1.5>
- [32] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Proc. Advances in Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 8024–8035.
- [33] S. Fang et al., "RepresentThemAll: A universal learning representation of bug reports," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 602–614.
- [34] S. Chaudhary, *CodeAlpaca: An Instruction-following LLaMA model for code generation*. GitHub repository, 2023. [Online]. Available: <https://github.com/sahil280114/codealpaca>
- [35] A. Gudibande et al., "The false promise of imitating proprietary LLMs," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.
- [36] Y. Bai et al., "Training a Helpful and Harmless Assistant with RLHF," arXiv preprint arXiv:2204.05862, 2022.
- [37] N. Jiang et al., "Impact of Code LMs on Automated Program Repair," arXiv preprint arXiv:2302.05020, 2023.
- [38] D. Sobania et al., "An analysis of the automatic bug fixing performance of ChatGPT," arXiv preprint arXiv:2301.08653, 2023.
- [39] D. Shrivastava et al., "Repository-Level Prompt Generation," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 31693–31715.
- [40] S. Fakhoury et al., "Generating Functionally Correct Code Edits from NL Issues," arXiv preprint arXiv:2304.03816, 2023.
- [41] J. Bai et al., "Qwen Technical Report," arXiv preprint arXiv:2309.16609, 2023.
- [42] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," in *Proc. Advances in Neural Inf. Process. Syst. (NeurIPS)*, 2023.
- [43] Z. Gou et al., "CRITIC: LLMs can self-correct with tool-interactive critiquing," arXiv preprint arXiv:2305.11738, 2023.
- [44] Y. Dong et al., "Self-collaboration Code Generation via ChatGPT," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 7, pp. 1–38, 2024.
- [45] S. Zhou et al., "CodeBERTScore: Evaluating Code Generation with Pre-trained Models," arXiv preprint arXiv:2302.05527, 2023.
- [46] A. Silva et al., "RepairLLaMA: Efficient representations and fine-tuned adapters for program repair," *IEEE Trans. Softw. Eng.*, vol. 51, no. 8, pp. 2366–2380, 2025.